SYSTEM DESIGN

뉴스 피드 · 채팅 · 검색어 자동완성

11–13장 핵심 설계

11장
비동기 팬아웃

12장
실시간 연결

13장
읽기 최적화

OVERVIEW

오늘의 흐름

01

뉴스 피드

발행·읽기·팬아웃

구조 이해 → 핵심 선택지 → 정리 → 토의 질문

02

채팅

웹소켓·저장·접속 상태

구조 이해 → 핵심 선택지 → 정리 → 토의 질문

03

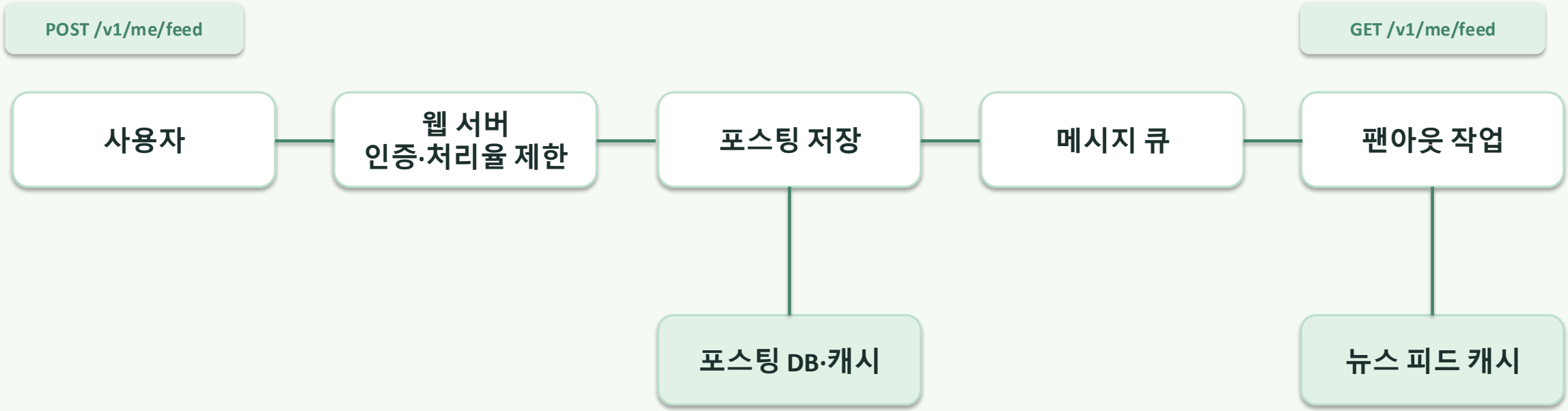
자동완성

트라이·수집·캐시

구조 이해 → 핵심 선택지 → 정리 → 토의 질문

각 장의 질문은 정답을 맞히는 문제가 아니라, 규모·지연·일관성·복구 전략의 우선순위를 함께 설명하는 토의입니다.

11장 | 피드는 “저장”과 “배포”를 나눈다



발행 경로

- 원본은 먼저 저장
- 친구별 피드 반영은 비동기 처리

읽기 경로

- 피드 ID를 캐시에서 조회
- 사용자·포스팅 정보를 조합

미디어

- 이미지·비디오는 CDN

11장 | 핵심 선택: 언제 피드를 만들까?

쓰기 시점 팬아웃

게시 직후 친구들의 피드 캐시를 갱신

- 읽기 지연이 짧고 실시간 반영
- 친구가 많은 사용자는 쓰기 비용·핫키 문제
- 비활성 사용자에게도 계산 자원 사용

읽기 시점 팬아웃

조회할 때 친구들의 게시물을 합침

- 비활성 사용자·핫키에 유리
- 피드를 읽는 시간이 길어짐
- 사용자·팔로어 규모에 따라 선택 변화

저장소 선택도 같은 문제다. 관계와 일관성이 중요한 원본은 SQL, <사용자 ID, 포스팅 ID>처럼 단순·빈번한 조회 데이터는 캐시나 NoSQL에 둘 수 있다.

NEWS FEED · SUMMARY

11장 정리 | 빠른 읽기를 위해 미리 계산한다

1

분리

원본 저장과 피드 배포를 분리

2

비동기

메시지 큐로 팬아웃 결합도 완화

3

관측

지연·캐시 적중률·큐 대기량 확인

확장 포인트

- 웹 계층은 무상태로 두어 수평 확장
- 복제본 읽기·샤딩·다중 데이터센터는 규모와 일관성 요구에 맞춰 선택
- 사용자 체감 지연을 중심으로 QPS·캐시·큐 지표를 함께 본다

토의 1 | 포스팅 저장에 성공하면 사용자에게 성공 응답을 보내도 될까?

포스팅 데이터베이스 저장은 완료되었다.
친구들의 뉴스 피드에 반영하는 작업은 메시지 큐를 통해 비동기로 처리한다.
메시지 큐에 이벤트를 넣는 과정에서 오류가 발생했다.

성공 응답을 보낸다

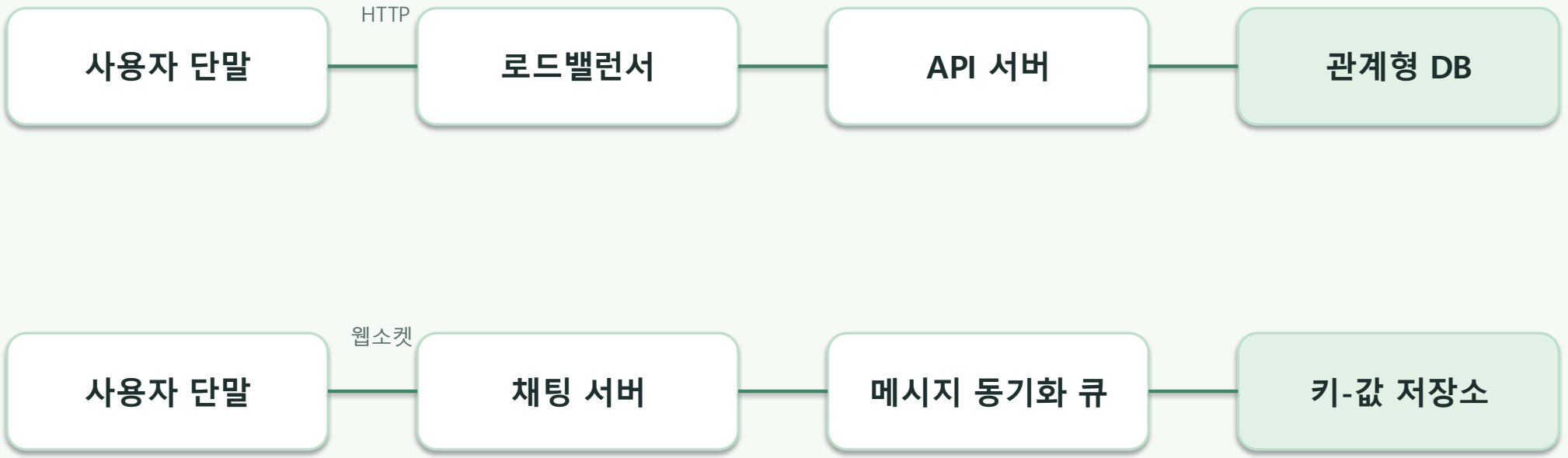
피드에 반영되지 않은 포스팅을
어떻게 찾아 복구할까?

성공 응답을 보내지 않는다

클라이언트 재시도로 생기는
중복 포스팅·피드를 어떻게 막을까?

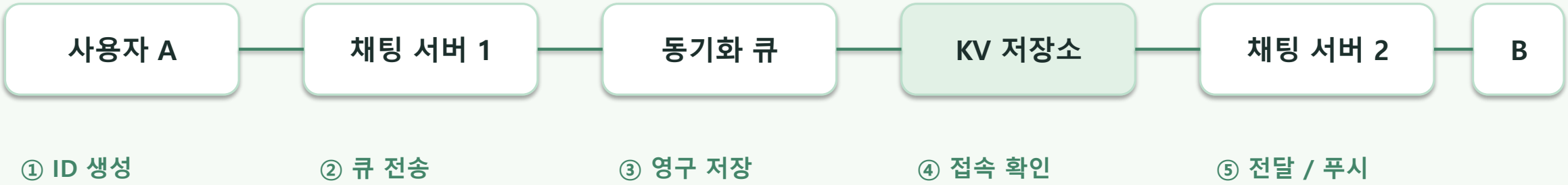
토의 키워드: 성공의 기준 · 재시도 · 멱등성 키 · <사용자 ID, 포스팅 ID> 중복 검사 · 실패 작업 모니터링

12장 | 실시간 기능과 일반 API를 분리한다



- HTTP: 로그인·프로필·그룹 관리
- 웹소켓: 메시지·접속 상태의 양방향 전송
- 관계형 DB: 사용자·친구·그룹
- KV 저장소: 대규모 채팅 이력과 빠른 접근

12장 | 메시지는 “저장”과 “전달” 상태가 다르다



다중 단말 동기화
cur_max_message_id

접속 상태
박동 이벤트 + last_active_at

소규모 그룹
수신자별 동기화 큐 복사

message_id는 고유하고 시간순 정렬 가능해야 하며, 재연결 후 누락 메시지를 찾는 기준이 된다.

CHAT · SUMMARY

12장 정리 | 지속 연결은 상태 관리 문제다

통신	저장	복구
일반 기능은 HTTP, 실시간은 웹소켓	관계형 원본 + KV 채팅 이력	재연결·ID 기반 동기화·푸시 알림

- 서비스 탐색은 로그인 시 적절한 채팅 서버를 선택
- 웹소켓 단절만으로 즉시 오프라인 처리하지 않고 박동으로 판단
- 그룹 규모가 커지면 수신자별 복사 비용과 접속 상태 전파 전략이 달라진다

토의 2 | 메시지가 저장되면 사용자에게 전송 성공으로 표시해도 될까?

사용자 A가 보낸 메시지는 키-값 저장소에 정상적으로 저장되었다.
사용자 B가 접속한 채팅 서버로 메시지를 전달하는 과정에서 오류가 발생했다.
사용자 A는 메시지 전송 결과를 기다리고 있다.

저장 직후 성공 표시

사용자 B에게 전달되지 않은 메시지는 어떻게 복구할까요?

수신 확인 후 성공 표시

사용자 B가 오프라인이거나 네트워크 상태가 좋지 않을 때 사용자 A는 계속 기다려야 할까요?

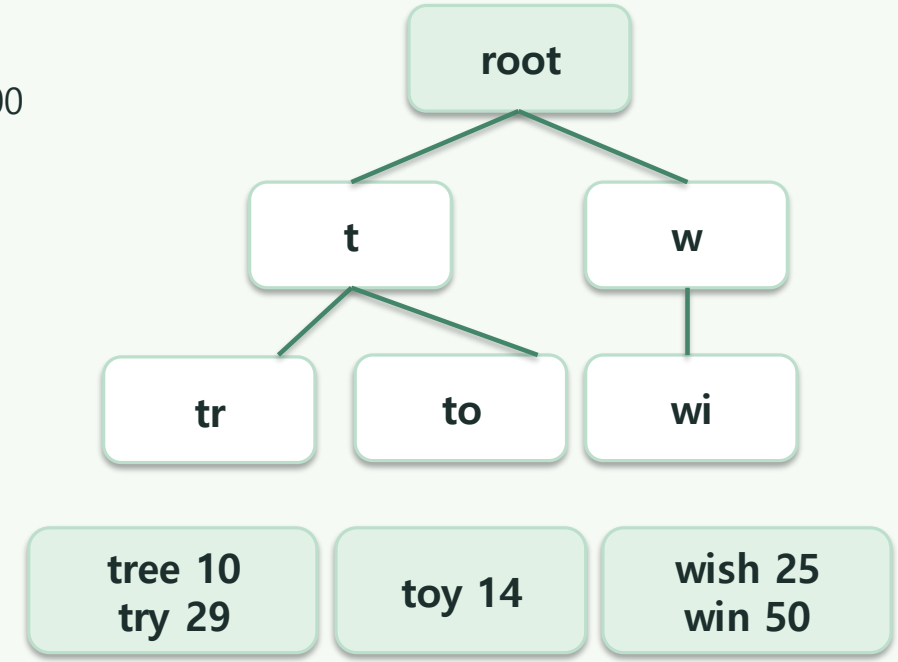
토의 키워드: 서버 접수·전달·읽음 상태 구분 · message_id · 재연결 동기화 · 중복 수신 · 푸시 알림

13장 | 자동완성은 읽기 경로를 미리 계산한다

요구사항

- 빠른 응답·연관성·정렬
- QPS 약 24,000 / 최대 48,000
- 상위 5개 검색어 반환
- 고가용성과 규모 확장성

트라이 예시



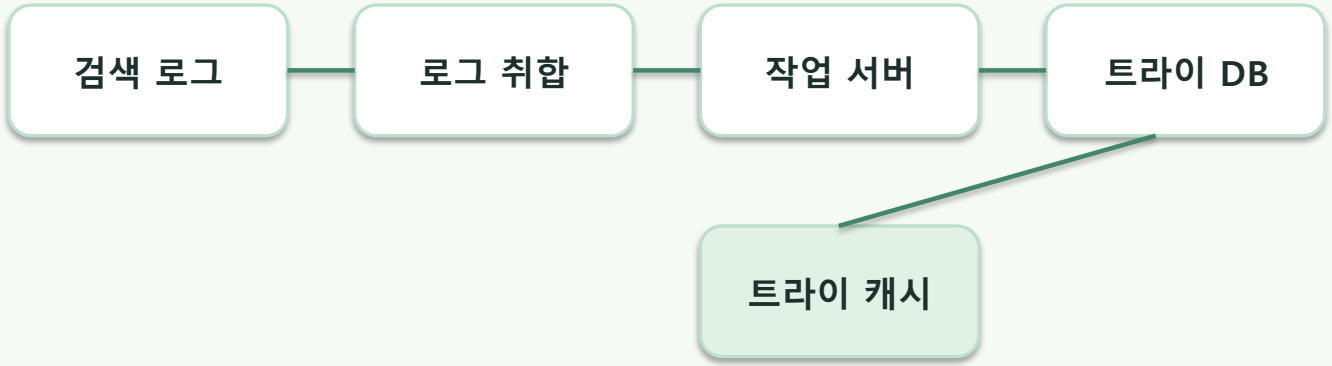
조회 절차

1. 접두어 노드 찾기
2. 하위 유효 노드 탐색
3. 빈도로 정렬
4. 상위 k개 반환

최적화: 접두어 길이 제한 + 각 노드에 인기 검색어 캐시

13장 | 수집과 질의를 분리하고 캐시한다

오프라인 수집·갱신 경로



온라인 질의 경로



- 브라우저 캐싱
- AJAX로 새로고침 없이 요청
- 캐시 미스 시 트라이 DB 조회

확장 포인트 | 국가별로 서로 다른 트라이 · 유니코드 저장 · 트라이를 CDN에 배치 · 첫 글자/과거 패턴 기반 샤딩 · 부적절한 검색어는 필터 계층에서 제외

13장 정리 | 빠른 응답은 최신성과 저장공간을 교환한다

트라이

접두어 기반 탐색으로
상위 k개 검색어 계산

캐시

각 노드의 인기 결과를 저장해
질의 시간을 O(1) 수준으로 단축

배치 갱신

로그를 취합해 주기적으로
새 트라이를 생성·교체

- 주간 갱신은 단순하지만 실시간 유행 반영이 어렵다
- 최근 검색어 가중치·샤딩·스트림 처리로 실시간성을 높일 수 있다
- 다국어·국가별 인기 검색어·유해 검색어 필터링은 서비스 정책과 연결된다

토의 3 | 인기 검색어가 갑자기 바뀌어도 매주 한 번만 트라이를 갱신해도 될까?

자동완성 질의는 트라이 캐시를 통해 빠르게 처리하며 작업 서버는 검색 로그를 취합하여 매주 한 번 새로운 트라이를 생성한다.
특정 이벤트로 새로운 검색어의 빈도가 갑자기 높아졌다.

배치 갱신 유지

새롭게 인기를 얻은 검색어가 자동완성 결과에 늦게 반영되는 문제를 허용할 수 있을까요?

실시간 스트림 반영

지속적으로 들어오는 검색 데이터를 처리하면서 트라이와 캐시를 어떻게 갱신할까요?

토의 키워드: 갱신 주기 · 최근성 가중치 · 샤딩 · 스트림 처리 · 국가별 트라이 · 유해 검색어 필터

CLOSING

세 시스템을 관통하는 한 가지 질문

“사용자에게 빠르게 보이는 결과”와 “정확하게 저장된 원본”을 어디서 분리할 것인가?

뉴스 피드

비동기 팬아웃
캐시된 피드 ID

채팅

지속 연결
저장·전달 상태 분리

자동완성

사전 계산된 트라이
캐시된 Top-k

공통 설계 언어

무상태 · 메시지 큐 · 캐시 · 고유 ID · 재시도 · 모니터링